

```
"""
review_verification_report.py

Standalone verification utility for the
Shield EPC AI pipeline's Stage 3
artifacts: Verification_Report.md and
Verification_Report.json.

This script performs no modification of any
project files. It only reads
the two report artifacts (expected in the
current working directory, or at
paths supplied on the command line) and
validates their schema, content,
and mutual consistency.

Usage:
    python review_verification_report.py
    python review_verification_report.py
path/to/Verification_Report.md
path/to/Verification_Report.json

Exit codes:
    0 - all checks passed
    1 - one or more checks failed
"""

import json
import os
import re
```

```

import sys
from datetime import datetime

DEFAULT_MD_PATH = "Verification_Report.md"
DEFAULT_JSON_PATH =
"Verification_Report.json"

REQUIRED_JSON_KEYS = ("status",
"generated_at", "recommendations")
PLACEHOLDER_MARKERS = ("TODO", "FIXME", "
<insert>")
KNOWN_PROVIDERS = ("groq", "openai")

def check_files_exist(md_path, json_path):
    """Verify that both the Markdown and
    JSON report files exist.

    Returns a list of error strings (empty
    if both files exist).
    """
    errors = []
    if not os.path.isfile(md_path):
        errors.append(f"Missing file:
{md_path}")
    if not os.path.isfile(json_path):
        errors.append(f"Missing file:
{json_path}")
    return errors

def load_json_report(json_path):
    """Load and parse the JSON report.

```

```

    Returns a tuple (data, errors). `data`
is None if the file could not
    be read or parsed as valid JSON.
    """
    errors = []
    try:
        with open(json_path, "r",
encoding="utf-8") as f:
            data = json.load(f)
        except json.JSONDecodeError as exc:
            errors.append(f"Invalid JSON in
{json_path}: {exc}")
            return None, errors
        except OSError as exc:
            errors.append(f"Could not read
{json_path}: {exc}")
            return None, errors

        if not isinstance(data, dict):
            errors.append(f"{json_path} does
not contain a JSON object at the top
level")
            return None, errors

    return data, errors

def validate_json_schema(data):
    """Validate required keys, types, and
values in the parsed JSON report.

    Returns a list of error strings (empty
if the schema is valid).
    """

```

```
errors = []

for key in REQUIRED_JSON_KEYS:
    if key not in data:
        errors.append(f"Missing key:
{key}")

    if "status" in data and
data.get("status") != "ok":
        errors.append(f"status must equal
'ok' (found: {data.get('status')!r})")

    if "recommendations" in data:
        recs = data.get("recommendations")
        if not isinstance(recs, list):
            errors.append("recommendations
must be a list")
        elif len(recs) == 0:
            errors.append("recommendations
must be a non-empty list")

    if "generated_at" in data:
        generated_at =
data.get("generated_at")
        if not isinstance(generated_at,
str) or not generated_at.strip():
            errors.append("generated_at
must be a non-empty string")
        else:
            # Lenient ISO-8601 check: try
to actually parse it rather than
            # pattern-matching, so valid
variations (offset vs 'Z' suffix,
            # with/without microseconds)
```

```

aren't rejected as malformed.
        normalized =
generated_at.strip()
        if normalized.endswith("Z"):
            normalized =
normalized[:-1] + "+00:00"
        try:

datetime.fromisoformat(normalized)
        except ValueError:
            errors.append(
                f"generated_at is not a
valid ISO-8601 timestamp: {generated_at!r}"
            )

        return errors

def load_markdown_report(md_path):
    """Read the Markdown report as text.

    Returns a tuple (content, errors).
    `content` is None if the file could
    not be read.
    """
    errors = []
    try:
        with open(md_path, "r",
encoding="utf-8") as f:
            content = f.read()
        except OSError as exc:
            errors.append(f"Could not read
{md_path}: {exc}")
        return None, errors

```

```

        return content, errors

def validate_markdown_content(md_content):
    """Validate the structural and content
    requirements of the Markdown report.

    Returns a list of error strings (empty
    if all checks pass).
    """
    errors = []

    if md_content is None or not
md_content.strip():
        errors.append("Markdown report is
empty")
        return errors

    if not re.search(r"^#\s+.+",
md_content, re.MULTILINE):
        errors.append("Markdown missing
report title (expected a top-level '#'
heading)")

    if not re.search(r"^#\s+Summary\s*$",
md_content, re.MULTILINE):
        errors.append("Markdown missing
Summary section")

    if not
re.search(r"^#\s+Recommendations\s*$",
md_content, re.MULTILINE):
        errors.append("Markdown missing
Recommendations section")

```

```

        if not any(provider in
md_content.lower() for provider in
KNOWN_PROVIDERS):
            errors.append("Markdown does not
mention a known provider name (Groq or
OpenAI)")

        lower_content = md_content
        for marker in PLACEHOLDER_MARKERS:
            if marker in lower_content:
                errors.append(f"Markdown
contains placeholder text: {marker}")

        return errors

def
extract_markdown_recommendations(md_content
):
    """Return the list of bullet-point
recommendation lines found under the
    '## Recommendations' section of the
Markdown report.

    Returns an empty list if the section is
missing or contains no bullets.
    """
    if not md_content:
        return []

    section_match = re.search(
        r"^##\s+Recommendations\s*$(.*?)
(^##\s+|\Z)",

```

```

        md_content,
        re.MULTILINE | re.DOTALL,
    )
    if not section_match:
        return []

    section_text = section_match.group(1)
    # Accept dash bullets, asterisk
    bullets, and numbered lists (e.g. "1. ").
    bullets = re.findall(r"^(?:[-
    *]|\d+\.)\s+.\+$", section_text,
    re.MULTILINE)
    return bullets

def
detect_markdown_provider_mentions(md_content):
    """Return the set of known provider
    names mentioned in the Markdown text."""
    if not md_content:
        return set()
    lower_content = md_content.lower()
    return {provider for provider in
    KNOWN_PROVIDERS if provider in
    lower_content}

def detect_json_provider(data):
    """Best-effort detection of a provider
    name referenced in the JSON report.

    Looks in common fields that might carry
    provider information. Returns

```



```

        None if no provider information is
        present, since the JSON schema does
        not guarantee a dedicated provider
        field.
        """
        if not data:
            return None

        candidate_fields = ("provider",
                             "report_provider", "input_from", "note")
        for field in candidate_fields:
            value = data.get(field)
            if isinstance(value, str):
                lower_value = value.lower()
                for provider in
KNOWN_PROVIDERS:
                    if provider in lower_value:
                        return provider

        return None

def cross_check_consistency(data,
md_content):
    """Cross-check consistency between the
    JSON and Markdown reports.

    Returns a list of error strings (empty
    if all checks pass).
    """
    errors = []

    if data is None or md_content is None:
        # Individual file/schema errors
        already reported elsewhere.

```

```

        return errors

        if data.get("status") != "ok":
            errors.append("Consistency check
skipped for status (JSON status is not
'ok')")

        json_recs = data.get("recommendations",
[])
        json_rec_count = len(json_recs) if
isinstance(json_recs, list) else 0
        md_rec_count =
len(extract_markdown_recommendations(md_con
tent))

        if md_rec_count < json_rec_count:
            errors.append(
                f"Recommendation count in
Markdown ({md_rec_count}) is less than "
                f"in JSON ({json_rec_count})"
            )

        json_provider =
detect_json_provider(data)
        md_providers =
detect_markdown_provider_mentions(md_conten
t)

        if json_provider is not None and
md_providers and json_provider not in
md_providers:
            errors.append(
                f"Provider mismatch: JSON
suggests '{json_provider}' but Markdown "
                f"mentions

```

```

{sorted(md_providers)}"
    )

    return errors

def run_checks(md_path, json_path):
    """Run all verification steps in order
    and collect all errors found.

    Returns a list of error strings. An
    empty list means every check passed.
    """
    all_errors = []

    # Step 1: file existence
    existence_errors =
check_files_exist(md_path, json_path)
    all_errors.extend(existence_errors)
    if existence_errors:
        # Can't proceed to content checks
without both files.
        return all_errors

    # Step 2: JSON schema validation
    json_data, json_load_errors =
load_json_report(json_path)
    all_errors.extend(json_load_errors)
    if json_data is not None:

all_errors.extend(validate_json_schema(json
_data))

    # Step 3: Markdown content validation

```

```

        md_content, md_load_errors =
load_markdown_report(md_path)
        all_errors.extend(md_load_errors)
        if md_content is not None:

all_errors.extend(validate_markdown_content
(md_content))

        # Step 4: cross-check consistency (only
if both loaded successfully)
        if json_data is not None and md_content
is not None:

all_errors.extend(cross_check_consistency(j
son_data, md_content))

        return all_errors

def print_report(errors):
    """Print a concise PASS/FAIL report
based on the collected errors."""
    if not errors:
        print("PASS")
        print("\u2713 JSON schema valid")
        print("\u2713 Markdown sections
present")
        print("\u2713 Recommendations
found")
        print("\u2713 Consistency checks
passed")
    else:
        print("FAIL")
        for error in errors:

```

```
print(f"\u2717 {error}")
```

```
def main():  
    """Entry point: resolve report paths,  
    run checks, print results, exit."""  
    md_path = sys.argv[1] if len(sys.argv)  
> 1 else DEFAULT_MD_PATH  
    json_path = sys.argv[2] if  
len(sys.argv) > 2 else DEFAULT_JSON_PATH  
  
    errors = run_checks(md_path, json_path)  
    print_report(errors)  
  
    sys.exit(0 if not errors else 1)  
  
if __name__ == "__main__":  
    main()
```